

Laboratorio 15 – Producto Punto con MPI

Curso: FS0432 – Física Computacional

Laboratorio: 15

Tema: Producto punto utilizando MPI

Estudiante / Carné: Jose Ricardo Montero Campos / C05005

Resumen

En este laboratorio se implementó un programa en C++ utilizando la biblioteca MPI (Message Passing Interface) para calcular el producto punto entre dos vectores de manera paralela. El programa inicializa los vectores únicamente en el proceso principal, distribuye los datos entre los procesos mediante envío y recepción de mensajes, calcula productos parciales y finalmente combina los resultados usando una reducción global con `MPI_Reduce`. Además, se midió el tiempo de ejecución para diferentes cantidades de procesos con el fin de analizar el desempeño de la paralelización.

Objetivos

Objetivo general

Implementar el cálculo paralelo del producto punto entre dos vectores utilizando la biblioteca MPI.

Objetivos específicos

- Inicializar dos vectores en el proceso principal.
 - Distribuir los datos entre los procesos mediante `MPI_Send` y `MPI_Recv`.
 - Calcular el producto punto local en cada proceso.
 - Obtener el producto punto global mediante `MPI_Reduce`.
 - Analizar el efecto del número de procesos sobre el tiempo de ejecución.
-

Fundamento teórico

El producto punto entre dos vectores (A) y (B) de longitud (N) se define como

$$A \cdot B = \sum_{i=0}^{N-1} A_i B_i.$$

Cuando el tamaño de los vectores es grande, esta operación puede paralelizarse dividiendo los datos entre varios procesos.

MPI (Message Passing Interface) es un estándar para programación paralela basado en el intercambio de mensajes entre procesos. En este laboratorio se utilizaron principalmente las siguientes funciones:

- **MPI_Init()**: inicializa el entorno MPI.
- **MPI_Comm_rank()**: obtiene el identificador del proceso.
- **MPI_Comm_size()**: obtiene el número total de procesos.
- **MPI_Send()**: envía datos a otro proceso.
- **MPI_Recv()**: recibe datos provenientes de otro proceso.
- **MPI_Reduce()**: combina los resultados parciales utilizando una operación global.
- **MPI_Finalize()**: finaliza el entorno MPI.

La paralelización consiste en repartir el trabajo entre los procesos disponibles para reducir el tiempo de ejecución. Cada proceso calcula únicamente una parte de la suma total y posteriormente todas las contribuciones son combinadas mediante una reducción global.

Metodología

Se utilizó un tamaño de vector

$N=128000$.

Los vectores fueron inicializados únicamente por el proceso con `rank = 0` utilizando

$$A_i = \sin(i)$$

y

$$B_i = 2\sin(i).$$

Posteriormente, el proceso principal dividió ambos vectores en bloques de igual tamaño y envió cada bloque al proceso correspondiente mediante `MPI_Send`, utilizando etiquetas diferentes para los vectores `A` y `B`.

Cada proceso recibió sus datos mediante `MPI_Recv` y calculó el producto punto correspondiente a su bloque local.

Una vez obtenidas las sumas parciales, se utilizó la función `MPI_Reduce` con la operación `MPI_SUM` para obtener el producto punto global en el proceso principal.

Finalmente, se midió el tiempo correspondiente al cálculo del producto punto utilizando la función `MPI_Wtime()`.

Resultados

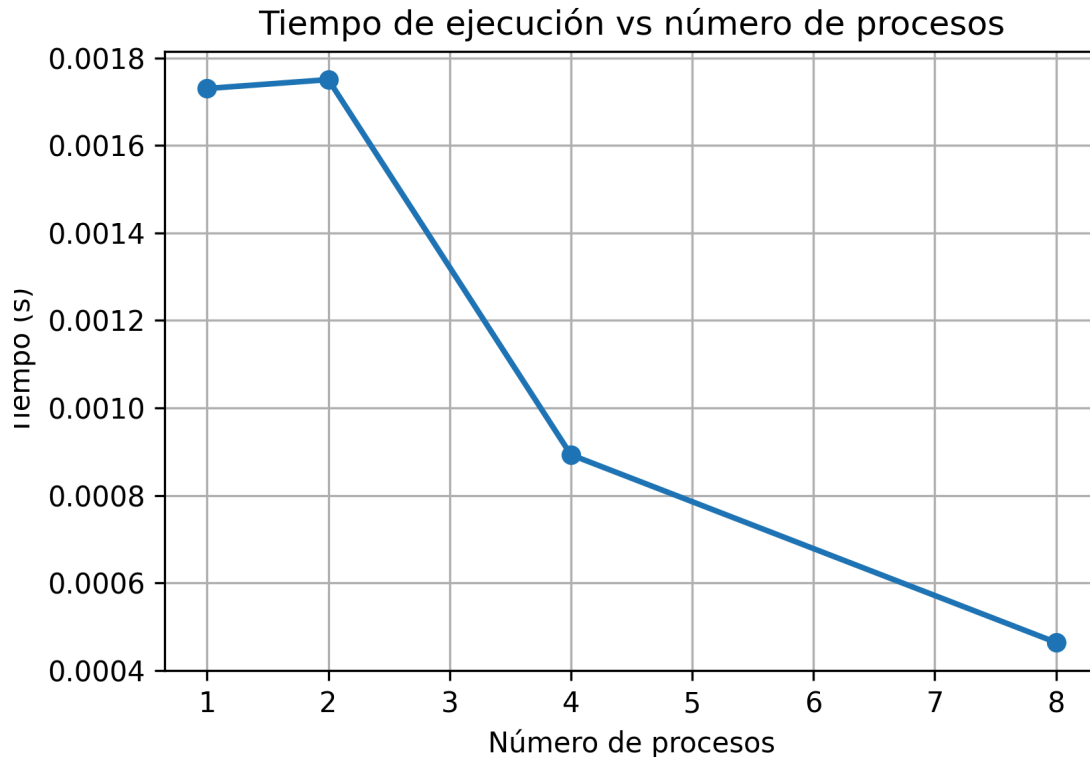
El producto punto obtenido fue

Número de procesos	Producto punto
1	128000
2	128000
4	128000
8	128000

Los tiempos medidos fueron

Procesos	Tiempo (s)
1	0.0017301
2	0.0017504
4	0.0008929
8	0.0004640

Gráfica de tiempos



Análisis de resultados

Los resultados muestran que el producto punto obtenido es el mismo independientemente del número de procesos utilizados, lo que confirma que la implementación paralela conserva la exactitud del cálculo.

Respecto al rendimiento, se observa que el tiempo de ejecución disminuye conforme aumenta el número de procesos, especialmente al pasar de cuatro a ocho procesos. Sin embargo, el tiempo obtenido con dos procesos fue ligeramente superior al registrado con un proceso. Este comportamiento es esperado en problemas relativamente pequeños, ya que el costo asociado a la creación de procesos, el envío de información y la sincronización puede superar el beneficio de dividir el trabajo.

En este laboratorio el tiempo medido corresponde únicamente al cálculo del producto punto. Si se incluyera el tiempo de comunicación entre procesos, la diferencia entre las ejecuciones sería aún más evidente.

Conclusiones

1. Se implementó correctamente el cálculo paralelo del producto punto utilizando MPI.
2. La distribución de datos mediante `MPI_Send` y `MPI_Recv` permitió que cada proceso trabajara únicamente sobre una fracción del problema.
3. La función `MPI_Reduce` simplifica la obtención del resultado global al combinar automáticamente las sumas parciales calculadas por todos los procesos.
4. El producto punto obtenido fue igual a 128000 para todas las ejecuciones, lo que confirma el correcto funcionamiento del programa.
5. La paralelización reduce el tiempo de ejecución cuando aumenta el número de procesos, aunque para problemas de tamaño pequeño el costo de comunicación puede limitar la mejora del rendimiento.